

Load libraries and set user directory paths.

```
In [ ]: ## Function to run shell commands in Google Colab with R kernel
shell_call <- function(command, ...) {
  result <- system(command, intern = TRUE, ...) # Runs the shell command and captures output
  cat(paste0(result, collapse = "\n")) # Prints the output to the console
}

## Function to Load multiple R packages
loadPackages = function(pkgs){
  myrequire = function(...){
    suppressWarnings(suppressMessages(suppressPackageStartupMessages(require(...))))
  }
  ok = sapply(pkgs, require, character.only=TRUE, quietly=TRUE) # Checks if each package is installed
  if (!all(ok)){
    message("There are missing packages: ", paste(pkgs[!ok], collapse=", ")) # Prints missing packages
  }
}

## Download the script to add R2U (fast package installation system)
download.file("https://github.com/eddelbuettel/r2u/raw/master/inst/scripts/add_cranapt_jammy.sh",
              "add_cranapt_jammy.sh")
Sys.chmod("add_cranapt_jammy.sh", "0755") # Change permissions to allow execution

## Run the script to set up R2U
shell_call("./add_cranapt_jammy.sh")

## Enable BSPM package manager for installing system-wide packages
bspm::enable()
options(bspm.version.check=FALSE)

## Remove the setup script to keep the workspace clean
shell_call("rm add_cranapt_jammy.sh")

## Define the list of packages to install
cranPkgs2Install = c("dplyr", "ggpubr", "Seurat", "cowplot",
                     "Rtsne", "hdf5r", "patchwork")

## Install all packages without prompting the user
install.packages(cranPkgs2Install, ask=FALSE, update=TRUE, quietly=TRUE)
```

```
In [2]: ## To simplify package loading, we created the loadPackages() function.
## But, if you don't have the function, you should use 'library(name_of_package)'
pkgs = c("Seurat", "dplyr", "patchwork") # List of packages to load
loadPackages(pkgs) # Load packages using previously defined function

## Define the directory where data will be stored and accessed
## IMPORTANT: The user must change "scw01" to match their actual working directory
```

Introduction

In this notebook, we will delve into the analysis of multi-modal single-cell data. Our focus will be on a non-small cell lung cancer (NSCLC) sample that was processed using the 10X 5' immune profiling technology. This advanced technology captures both RNA and T cell receptor sequences for each individual cell, providing a comprehensive view of the cellular landscape.

You can download the dataset from the 10x Genomics website [here](#).

- Before we proceed, please take a moment to read the detailed description of the sample processing and early analysis steps provided on the webpage. This will give you valuable context and insights into the experimental setup and the quality of the data.

TCR Sequence

The single-cell TCR sequencing approach enables high-resolution analysis of T-cell receptor diversity, identifying specific clonotypes and their roles in the immune system. Unlike bulk sequencing, which combines information from multiple cells, the single-cell method preserves the individual identity of each T cell, allowing for the correlation between transcriptomic profile and TCR repertoire.

NOTE:

In this notebook, we will briefly go through the key steps of single-cell analysis. Since this module focuses on TCR sequence analysis, we will provide an overview of the main processes. For more detailed explanations of each step, please refer to the other available modules.

Read in raw gene counts and metadata

```
In [4]: # Download a filtered gene-barcode matrix from 10X Genomics
# This command uses curl to download a file from the internet. (-O) Saves the file with the same name
shell_call("curl -O https://cf.10xgenomics.com/samples/cell-vdj/5.0.0/vdj_v1_hs_nsclc_multi_5g")

## Extract the downloaded file (decompress the dataset)
# tar -xf instructs tar to extract (-x) the specified file (-f) and decompress it.
shell_call("tar -xf vdj_v1_hs_nsclc_multi_5gex_t_b_count_filtered_feature_bc_matrix.tar.gz")

In [6]: # Read the filtered feature-barcode matrix into a matrix object
# This command reads the filtered feature-barcode matrix from a 10X Genomics dataset located in the
# directory and assigns the resulting data to the variable counts.
counts <- Read10X("filtered_feature_bc_matrix/")
```

Create the Seurat Object

Next, we will use the count matrix and metadata to create a Seurat object. The Seurat object acts as a comprehensive container that holds not only the raw count data and metadata but also any downstream analysis results, such as PCA (Principal Component Analysis) and clustering outcomes. This centralized container allows for streamlined data manipulation and analysis, providing an organized structure for our scRNA-Seq dataset.

By creating a Seurat object, we can efficiently manage and analyze our data, making it easier to perform complex operations and visualize the results.

```
In [7]: # Create a Seurat object using the raw count matrix
seurat.raw <- CreateSeuratObject(counts = counts)

# Show the contents of Seurat object
seurat.raw
```

An object of class Seurat
36601 features across 7092 samples within 1 assay
Active assay: RNA (36601 features, 0 variable features)
1 layer present: counts

Exploring the Seurat Object

The first step in our analysis is to familiarize ourselves with the dataset now stored in the Seurat object. Seurat provides powerful tools that allow us to explore and visualize our count data alongside the associated metadata. This enables us to gain a comprehensive understanding of our scRNA-Seq data.

By leveraging Seurat's capabilities, we can perform various exploratory analyses, such as:

- Inspecting the Distribution of Gene Expression: We can visualize the distribution of gene expression levels across cells to identify highly expressed genes and detect any potential outliers.
- Exploring Metadata: We can explore the metadata associated with our cells, such as cell type annotations, sample origins, and experimental conditions, to understand the context and characteristics of our dataset.
- Identifying Variable Genes: We can identify variable genes that exhibit significant variability across cells, which are often of interest for downstream analyses like clustering and differential expression.
- Visualizing Data: We can create visualizations like violin plots, feature plots, and heatmaps to explore the expression patterns of specific genes and compare different cell groups.

By conducting these initial explorations, we set the stage for more advanced analyses, such as dimensionality reduction, clustering, and differential expression analysis. This foundational step is crucial for understanding the nuances of our dataset and making informed decisions throughout our analysis pipeline.

```
In [8]: # How many cells and genes do we currently have?
print(paste0("The number of genes is ", dim(seurat.raw)[1], " and the number of cells is ", di

# The print() function displays the result in the console.
# The dim() function returns the dimensions of an object, usually a matrix or data frame.
# In the case of Seurat objects, the gene expression count matrix has genes as rows and cells
# The paste0() function concatenates (joins) strings without adding any spaces between them.
# The sentences inside the parentheses (") will also be printed
```

```
[1] "The number of genes is 36601 and the number of cells is 7092"
```

```
In [ ]: # View a subset (a slice) of the count matrix
# Remember: rows are genes, columns are cells/barcodes)
GetAssayData(seurat.raw, layer = "counts")[8:10,13:14]
```

NOTE:

The dots('.') reflect a zero value. The count table is stored in sparse matrix format which explicitly stores only non zero values to save space.

```
In [9]: ## Display metadata columns available in the Seurat object
# What metadata columns are available in the Seurat object?
print(colnames(seurat.raw@meta.data))

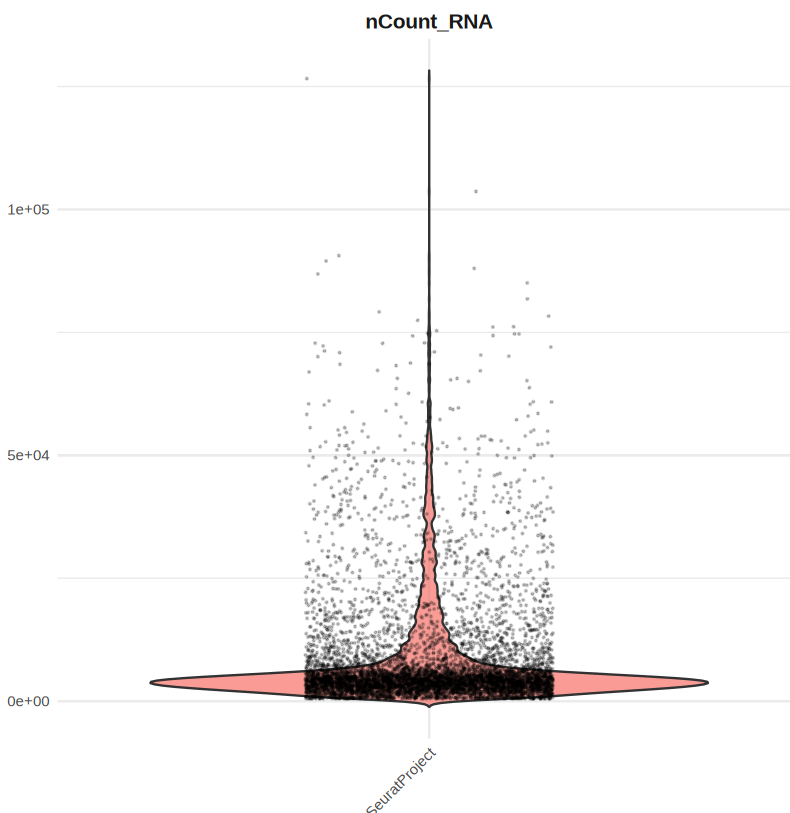
# (@) Accesses the meta.data slot of the seurat.raw object.
# The meta.data slot typically contains metadata associated with cells, such as cell type, sam
# The colnames() function retrieves the column names of the metadata data frame stored in the
```

```
[1] "orig.ident"    "nCount_RNA"    "nFeature_RNA"
```

```
In [19]: # Create a violin plot showing the distribution of number of UMIs per cell

options(repr.plot.width=7, repr.plot.height=7) # This command sets the width and height of the
VlnPlot(seurat.raw, features = c("nCount_RNA"),y.max=2e4)

# creates a violin plot
# seurat.raw: The Seurat object containing your single-cell data.
# features = c("nCount_RNA"): It is plotting the number of RNA molecules (nCount_RNA) for each
# y.max=2e4: Sets the maximum value for the y-axis to 20,000 (2e4 is scientific notation for 2
```



Quality Control

We have already observed that two of the primary quality control (QC) metrics, such as number of UMIs (Unique Molecular Identifiers) and number of genes detected per cell, were automatically computed by Seurat. The next important QC metric to consider is the percentage of mitochondrial genes. Mitochondrial genes can indicate cell stress or apoptosis, so it's crucial to monitor their expression levels.

To calculate the percentage of mitochondrial genes, we will use Seurat's `PercentageFeatureSet` method. This method calculates the percentage of UMIs originating from genes that match a specified pattern. For mitochondrial genes, we typically look for genes starting with "MT-" (common prefix for mitochondrial genes).

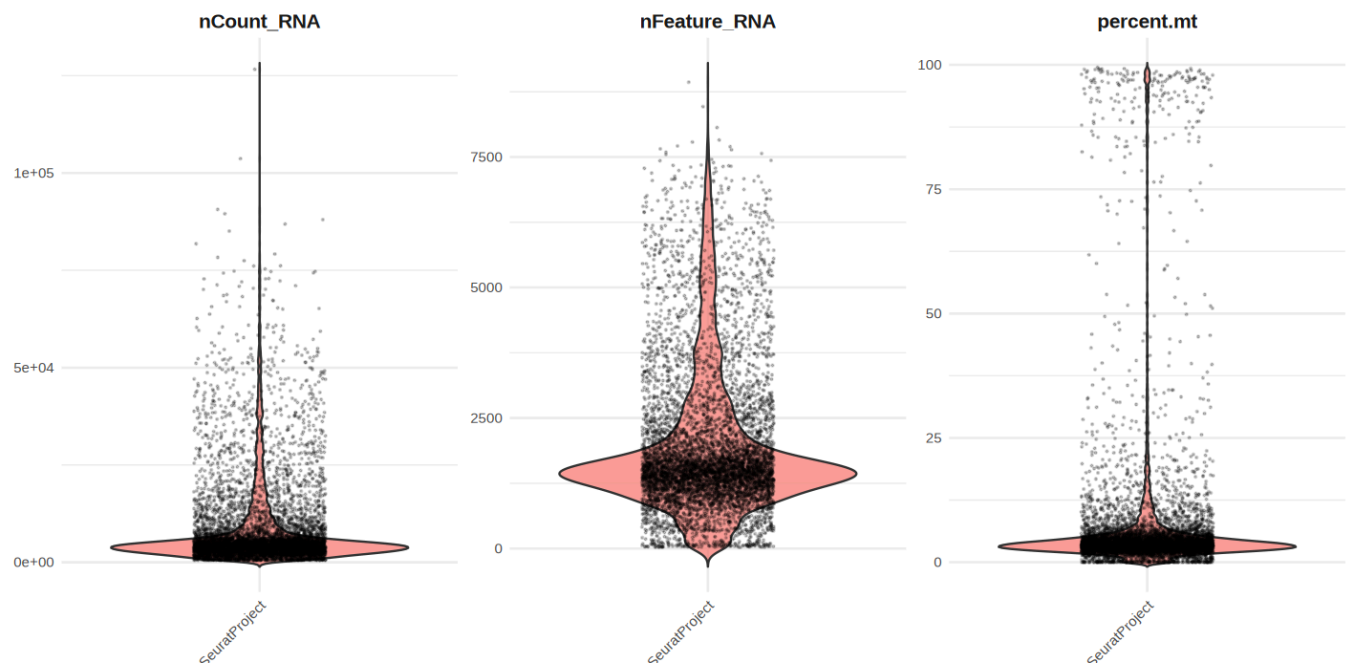
```
In [20]: # Human mitochondrial gene names start with "MT-" so we'll calculate the percentage of genes m
# This function calculates the percentage of counts for features (genes) that match a given pa
# Also creates a new metadata column in the seurat.raw object named percent.mt and assigns the
seurat.raw[["percent.mt"]] <- PercentageFeatureSet(seurat.raw, pattern = "^MT-")

# Now we can see that the % mitochondrial gene expression has been calculated for each cell
head(seurat.raw$percent.mt) # Display the first few rows of the calculated mitochondrial perce
```

```
AAACCTGAGCTAGTCT-1: 20.463428252373 AAACCTGAGGGCACTA-1: 3.34990840094216
AAACCTGAGTACGTTC-1: 2.87994991391454 AAACCTGAGTCCGGTC-1: 6.11979166666667
AAACCTGCACCAGGTC-1: 3.17273795534665 AAACCTGCACCTCGTT-1: 2.27431400342734
```

```
In [22]: ## Violin plot for three quality metrics: UMI count, gene count, mitochondrial gene percentage
options(repr.plot.width=12, repr.plot.height=6)
VlnPlot(seurat.raw, features = c("nCount_RNA", "nFeature_RNA", "percent.mt"))

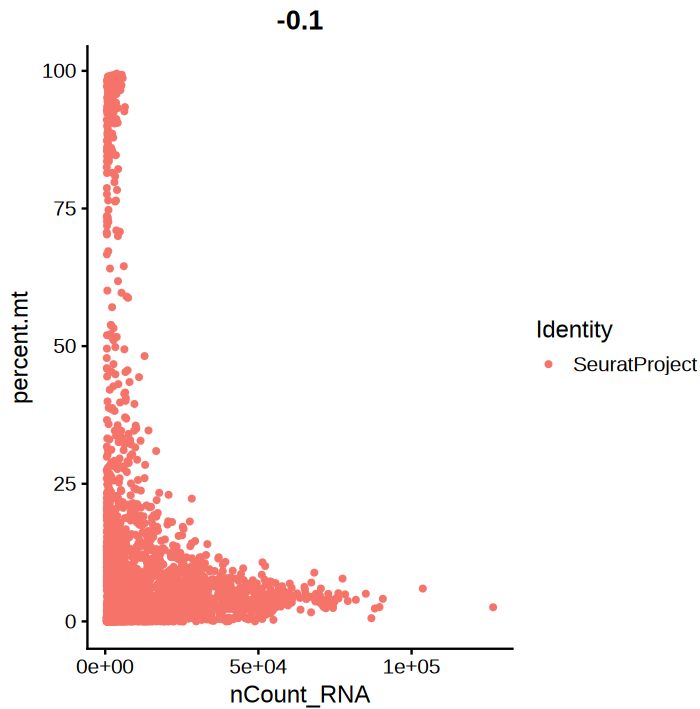
# We can visualize all three of the cell quality metrics together using Seurat's VlnPlot metho
```



It is often helpful to visualize these QC metrics in conjunction, as cells that are outliers in multiple dimensions, are more likely to be low quality cells. Seurat's FeatureScatter creates a scatter plot of two given columns from our metadata.

```
In [23]: ## Scatter plot: total UMI counts vs percentage of mitochondrial genes
options(repr.plot.width=6, repr.plot.height=6)
FeatureScatter(seurat.raw, feature1 = "nCount_RNA", feature2 = "percent.mt")

# Here we visualize the number of UMI vs % mito genes
```



```
In [24]: # Once we've visualized the metrics we can select the thresholds that we want to use to filter
# We use R's subset method to filter
seurat.raw <- subset(
  seurat.raw,
  subset = ## Apply filtering criteria:
    nFeature_RNA > 200 & # Remove cells with fewer than 200 genes
    nCount_RNA > 400 & # Remove cells with fewer than 400 UMIs
    nFeature_RNA < 6000 & # Remove cells with more than 6000 genes (potential doublets)
    percent.mt < 40) # Remove cells with more than 40% mitochondrial RNA (Low-quality)
```

Normalizing Data in Seurat

After removing unwanted cells from the dataset, the next step is to normalize the data. Normalization is essential to adjust for differences in sequencing depth across cells, ensuring meaningful comparisons.

1. Normalization Method: LogNormalize

- The most common normalization method in Seurat is the global-scaling normalization method called "LogNormalize." This method involves three main steps:

2. Normalization by Total Expression:

- For each cell, the method calculates the total expression (the sum of counts for all genes).
- Each gene expression value is divided by the total expression of the cell to adjust for differences in sequencing depth.

3. Multiplication by a Scaling Factor:

- The normalized values are then multiplied by a scaling factor (10,000 by default). This helps bring the values to a more convenient and interpretable scale.

4. Logarithmic Transformation:

- Finally, the normalized values are log-transformed using the natural logarithm. The log transformation stabilizes variance and reduces the impact of outliers, making the data distribution more symmetric.

5. Data Storage:

- The original raw counts are stored in `seurat.raw[["RNA"]]`@counts. This slot contains the unnormalized count data for each gene in each cell.
- The new normalized data are stored in `seurat.raw[["RNA"]]`@data. This slot contains the normalized and log-transformed expression values for each gene in each cell.

```
In [25]: # Normalize data using LogNormalization
seurat.raw <- NormalizeData(seurat.raw, normalization.method = "LogNormalize", scale.factor =
```

Normalizing layer: counts

Do standard variable genes discovery, Scaling, PCA, Clustering, and UMAP

```
In [28]: # Identify the most variable genes
seurat.raw <- FindVariableFeatures(seurat.raw, selection.method = "vst", nfeatures = 2000)
# Scale the data
seurat.raw <- ScaleData(seurat.raw, features = VariableFeatures(seurat.raw), do.scale = T, do.
# Run PCA (Principal Component Analysis)
seurat.raw <- RunPCA(seurat.raw, features = VariableFeatures(seurat.raw))
# Find cell neighbors
seurat.raw <- FindNeighbors(seurat.raw, dims = 1:20, k.param = 20)
# Identify clusters at low resolution
seurat.raw <- RunUMAP(seurat.raw, dims = 1:20, reduction = "pca", seed.use = 1)

# Find clusters and use a low resolution (0.1 is a good start) so that we can easily identify
seurat.raw <- FindClusters(seurat.raw, resolution = 0.1)
```

```
02:31:36 Commencing smooth kNN distance calibration using 1 thread
with target n_neighbors = 30

02:31:36 Initializing from normalized Laplacian + noise (using RSpectra)

02:31:36 Commencing optimization for 500 epochs, with 283724 positive edges

02:31:36 Using rng type: pcg

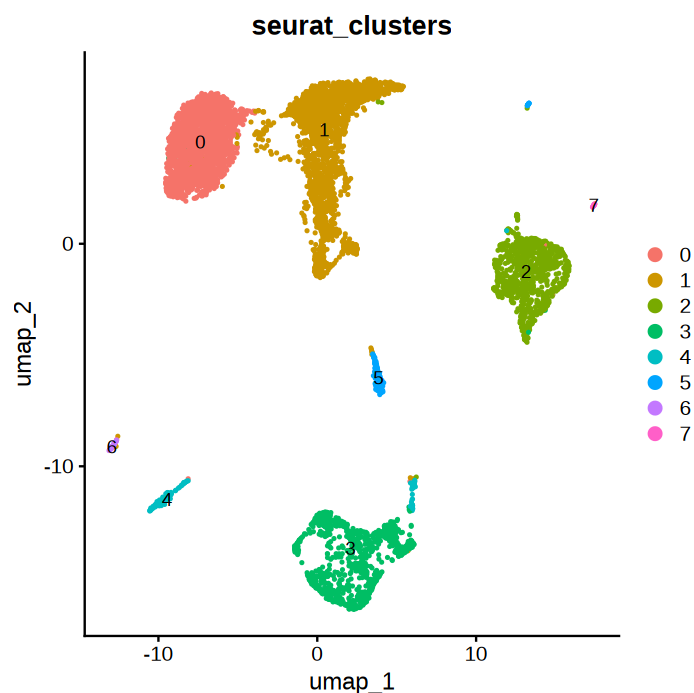
02:31:42 Optimization finished
```

Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 6654
Number of edges: 236025

Running Louvain algorithm...
Maximum modularity in 10 random starts: 0.9741
Number of communities: 8
Elapsed time: 0 seconds

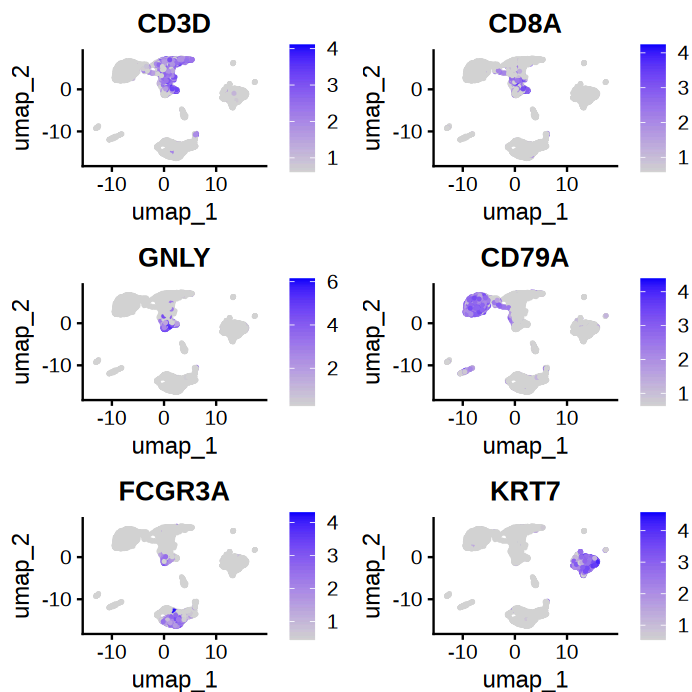
```
In [29]: ## Visualize UMAP with cluster labels
DimPlot(seurat.raw, reduction = "umap", label = T, group.by = "seurat_clusters")
```



Look at the feature plot of common cell type markers to figure out which cluster is T cells

- T cell genes: CD3D, CD8A, GNLY
- B cell gene: CD79A
- Myeloid cell gene: FCGR3A
- Epithelial (lung) cell gene: KRT7

```
In [30]: # Feature plot of specific marker genes to identify cell types
FeaturePlot(seurat.raw, features = c("CD3D", "CD8A", "GNLY", "CD79A", "FCGR3A", "KRT7"), min.cu
```

TCR sequence integration

In this section we will read in files generated by the 10X CellRanger software which describe the TCR sequences found in individual cells.

```
In [33]: #Download files with curl
shell_call("curl -O https://cf.10xgenomics.com/samples/cell-vdj/5.0.0/vdj_v1_hs_nsclc_multi_5g")
shell_call("curl -O https://cf.10xgenomics.com/samples/cell-vdj/5.0.0/vdj_v1_hs_nsclc_multi_5g")

# Read in TCR information for each cell
tcr <- read.csv("vdj_v1_hs_nsclc_multi_5gex_t_b_vdj_t_all_contig_annotations.csv")

# Read in clonotype info
# many cells can share the same clonotype, or TCR sequence
clono <- read.csv("vdj_v1_hs_nsclc_multi_5gex_t_b_vdj_t_clonotypes.csv")

# Remove the -1 at the end of each barcode.
tcr$barcode <- gsub("-1", "", tcr$barcode)

# Filter to keep only the first line of each cell barcode
# so that we only analyze one clonotype for each cell.
tcr <- tcr[!duplicated(tcr$barcode), ]

# Also remove the -1 at the end of the line of the Seurat object
seurat.raw = RenameCells(seurat.raw, new.names = gsub("-1", "", colnames(seurat.raw)))

# Only keep the barcode and clonotype columns.
tcr <- tcr[,c("barcode", "raw_clonotype_id")]

# Adjust the name from "raw_clonotype_id" to "clonotype_id" so
names(tcr)[names(tcr) == "raw_clonotype_id"] <- "clonotype_id"

# Merge the TCR and clonotype tables so we have TCR amino acid sequence for each cell
```

```
tcr <- merge(tcr, clono[, c("clonotype_id", "cdr3s_aa")])

# Reorder so barcodes are first column
tcr <- tcr[, c(2,1,3)]

# Set them as rownames
rownames(tcr) <- tcr[,1]

# Remove the unnecessary extra column of barcodes
tcr[,1] <- NULL

# Add to the Seurat object's metadata.
seurat.raw <- AddMetaData(object=seurat.raw, metadata=tcr)

# Confirm we have TCR information in the metadata
head(seurat.raw@meta.data)
```

A data.frame: 6 × 8

	orig.ident	nCount_RNA	nFeature_RNA	percent.mt	RNA_snn_res.0.1	seurat_cluster
	<fct>	<dbl>	<int>	<dbl>	<fct>	
AAACCTGAGCTAGTCT	SeuratProject	3582	1197	20.463428	2	
AAACCTGAGGGCACTA	SeuratProject	3821	1416	3.349908	0	
AAACCTGAGTACGTTC	SeuratProject	6389	1799	2.879950	0	
AAACCTGAGTCCGGTC	SeuratProject	3072	1125	6.119792	0	
AAACCTGCACCAGGTC	SeuratProject	9361	2661	3.172738	1	
AAACCTGCACCTCGTT	SeuratProject	47267	5607	2.274314	2	

```
In [34]: # Generate a histogram of clone frequencies from clono table
# What is a good threshold to distinguish expanded from non-expanded clones?
barplot(table(clono$frequency), xlab="Number of cells in clone")

# The barplot() function creates a bar plot.
# The table() function creates a contingency table of the counts at each unique value of clono
# clono$frequency accesses the frequency column in the clono data frame

# Identity expanded clones, flag in metadata, and label in UMAP
# How many expanded clones are there? Try changing the threshold and compare the results
Nexpand = 1
# This is the threshold we will use to distinguish expanded from non-expanded clones
length(which(clono$frequency > 1))
# which(clono$frequency > 1): Returns the indices of the rows where the frequency is greater than 1
# length(which(clono$frequency > 1)): Counts the number of elements (clones) that have a frequency greater than 1

expanded_clones = clono$clonotype_id[clono$frequency > 1]
# clono$clonotype_id[clono$frequency > 1]: Selects the clonotype_id values where the frequency is greater than 1
# expanded_clones: Stores the clonotype IDs of the expanded clones in a new variable.

# Add a new metadata column indicating which cells are part of an expanded clone
```

```

seurat.raw = AddMetaData(seurat.raw, metadata = rep("no", ncol(seurat.raw)), col.name = "TCR_expanded")
# This command adds a new metadata column to the seurat.raw Seurat object.
# in this new metadata where every cell is "no", and then changing the value for the expanded

seurat.raw@meta.data$TCR_expanded[seurat.raw@meta.data$clonotype_id %in% expanded_clones] = "yes"
# This command updates the TCR_expanded metadata column.
# seurat.raw@meta.data$TCR_expanded: Accesses the TCR_expanded column in the metadata.
# seurat.raw@meta.data$clonotype_id %in% expanded_clones: Checks which cells have clonotype_id
# = "yes": Sets the value to "yes" for cells that belong to expanded clones

# Look at the UMAP to see if expanded clones have similar gene expression
DimPlot(seurat.raw, reduction = "umap", group.by = "TCR_expanded", label = T)

# Do expanded T cells have different genes expressed compared to non-expanded T cells?
# To do ask this question, let's first separate the T cells from the rest of the object
# Check your clustering to see which one looks like it contains T cells

Idents(seurat.raw) = "seurat_clusters" # This means that subsequent operations will use the cl
seurat.t = subset(seurat.raw, idents = "1") #is the new Seurat object containing only the cell
seurat.t # Check the number of samples (cells) to see how many T cells we have to work with

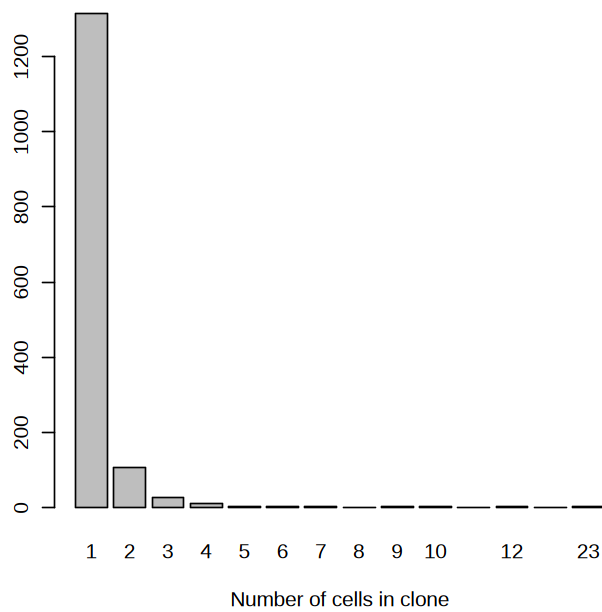
Idents(seurat.t) = "TCR_expanded" # This command sets the active identity class in the seurat.
deg_expanded = FindMarkers(seurat.t, ident.1 = "yes", ident.2 = "no", logfc.threshold = 0.25, min.pct
# This command identifies differentially expressed genes between two groups of cells
# Those marked as "yes" in the TCR_expanded metadata and those marked as "no".
# logfc.threshold: Minimum log2 fold-change threshold for identifying differentially expressed
# min.pct: Minimum percentage of cells in which the gene is detected.

# Visualize the expression of the top DE genes for each annotated cell subset.
top30genes <- deg_expanded %>% filter(avg_log2FC > 0) %>% top_n(30, avg_log2FC)
# deg_expanded %>% filter(avg_log2FC > 0): This filters the differentially expressed genes (de
# to include only those with a positive average log2 fold-change (avg_log2FC > 0).
# top_n(30, avg_log2FC): From the filtered genes, this selects the top 30 genes with the highe

genes <- rownames(top30genes)
seurat.t <- ScaleData(seurat.t, features = genes, do.center = T, do.scale = T)
# Scales the expression data for the selected genes (features = genes) in the seurat.t object.
# do.center = T: Centers the data by subtracting the mean expression for each gene.
# do.scale = T: Scales the data by dividing by the standard deviation for each gene

DoHeatmap(seurat.t, features = genes)
# Creates a heatmap of the expression data

```



An object of class Seurat
36601 features across 2035 samples within 1 assay
Active assay: RNA (36601 features, 2000 variable features)
3 layers present: counts, data, scale.data
2 dimensional reductions calculated: pca, umap

Centering and scaling data matrix

